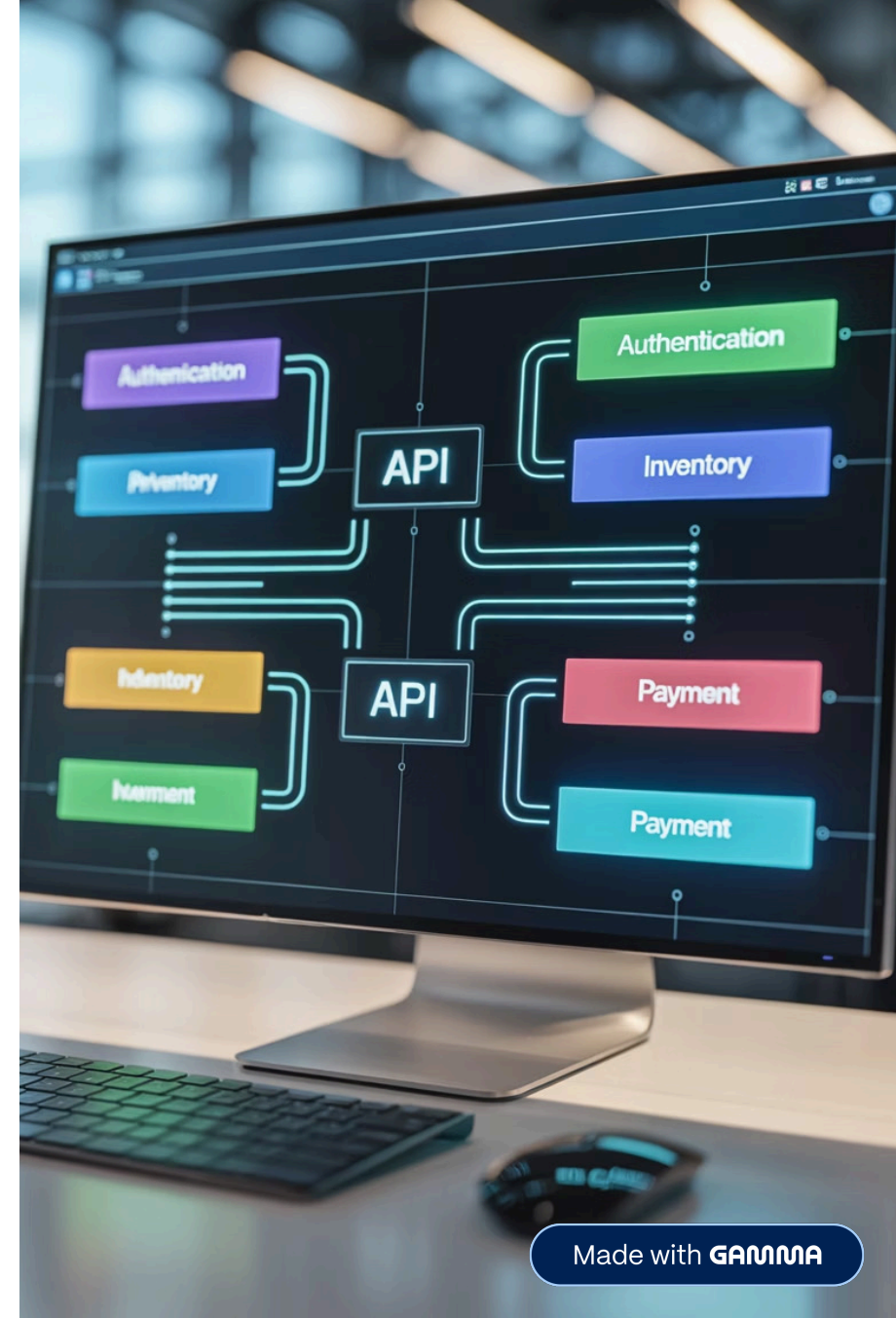


Kiến trúc microservices

Bản chất, chiến lược phân rã và ứng dụng thực tế



Nội dung trình bày

01

Bản chất và mục đích của microservices

Định nghĩa, so sánh với mô hình nguyên khối, ghép nối lỏng lẻo và triển khai độc lập

02

Chiến lược phân rã hệ thống

Phân rã theo năng lực kinh doanh, bối cảnh giới hạn và quyền sở hữu dữ liệu

03

Áp dụng

Thiết kế dịch vụ, ranh giới, hạ tầng hỗ trợ và quản trị

04

Phân tích ưu nhược điểm

Lợi ích, hạn chế và biện pháp giảm thiểu rủi ro

05

Trường hợp sử dụng thực tế

Amazon, Netflix, Uber, Etsy và ngành tài chính

Bản chất của microservices

"Các dịch vụ nhỏ, độc lập và được ghép nối lỏng lẻo, mỗi dịch vụ tự chứa logic và dữ liệu, chịu trách nhiệm thực hiện một khả năng kinh doanh trong bối cảnh giới hạn"

- Microsoft



- Chia hệ thống thành tập hợp các dịch vụ nhỏ, tự trị
- Mỗi dịch vụ là một thành phần riêng biệt
- Được xây dựng, chạy và triển khai độc lập
- Mỗi dịch vụ tự chứa logic và dữ liệu
- Chịu trách nhiệm cho một khả năng kinh doanh cụ thể

Monolithic
Architecture

BI

Microservices
Architecture.

7E in

Microservices so với mô hình nguyên khối

Mô hình nguyên khối (Monolithic)

- Gộp toàn bộ chức năng vào một mã nguồn lớn
- Sử dụng một cơ sở dữ liệu chung
- Khó hiểu, kiểm thử và mở rộng khi phình to
- Triển khai đồng thời toàn bộ ứng dụng
- Thay đổi nhỏ cũng cần build lại toàn bộ

Mô hình microservices

- Chia thành nhiều dịch vụ nhỏ, độc lập
- Mỗi dịch vụ có cơ sở dữ liệu riêng
- Dễ hiểu, kiểm thử và mở rộng từng phần
- Triển khai độc lập từng dịch vụ
- Thay đổi nhỏ được triển khai nhanh chóng

Mục đích của microservices



Tăng tốc độ ra mắt

Triển khai nhanh chóng các tính năng mới mà không ảnh hưởng đến toàn bộ hệ thống



Cải thiện khả năng mở rộng

Mở rộng từng thành phần theo nhu cầu thay vì toàn bộ ứng dụng



Tự chủ cho các nhóm

Cho phép các nhóm nhỏ làm việc độc lập, tự chủ về công nghệ và quy trình



Cách ly lỗi

Nếu một dịch vụ gặp sự cố, nó không làm sập toàn bộ hệ thống

Loose coupling

Các dịch vụ không chia sẻ nội bộ và giao tiếp qua API được định nghĩa rõ ràng:

- Mỗi dịch vụ **sở hữu dữ liệu của mình** và chỉ cung cấp dữ liệu qua API
- Mẫu **"Database per Service"** - dữ liệu bền vững của mỗi dịch vụ phải riêng tư
- Giảm phụ thuộc giữa các nhóm phát triển
- Cho phép thay đổi schema, công nghệ hoặc logic mà không ảnh hưởng tới dịch vụ khác



Triển khai độc lập

"Kiến trúc microservices gồm hai hoặc nhiều thành phần triển khai độc lập, ghép nối lỏng lẻo"

- Microservices.io



Kho mã riêng

Mỗi dịch vụ có kho mã nguồn riêng biệt



Pipeline CI/CD riêng

Mỗi dịch vụ có quy trình build, test và triển khai riêng



Triển khai độc lập

Có thể triển khai hoặc rollback một dịch vụ mà không phải build lại toàn bộ hệ thống

Nhờ triển khai độc lập, bạn có thể sửa lỗi hoặc thêm tính năng mà không dừng toàn bộ ứng dụng.

Chiến lược phân rã hệ thống

Phân rã theo năng lực kinh doanh (Business Capability)

Mỗi dịch vụ đại diện cho một khả năng kinh doanh và do một đội nhỏ chịu trách nhiệm

Phân rã theo bối cảnh giới hạn (Bounded Context/Subdomain)

Dựa trên Domain-Driven Design (DDD), mỗi dịch vụ tương ứng với một subdomain của miền ứng dụng

Phân rã theo quyền sở hữu dữ liệu

Microservices sở hữu dữ liệu của mình và không chia sẻ trực tiếp, tuân theo mẫu "Database per Service"



Phân rã theo năng lực kinh doanh

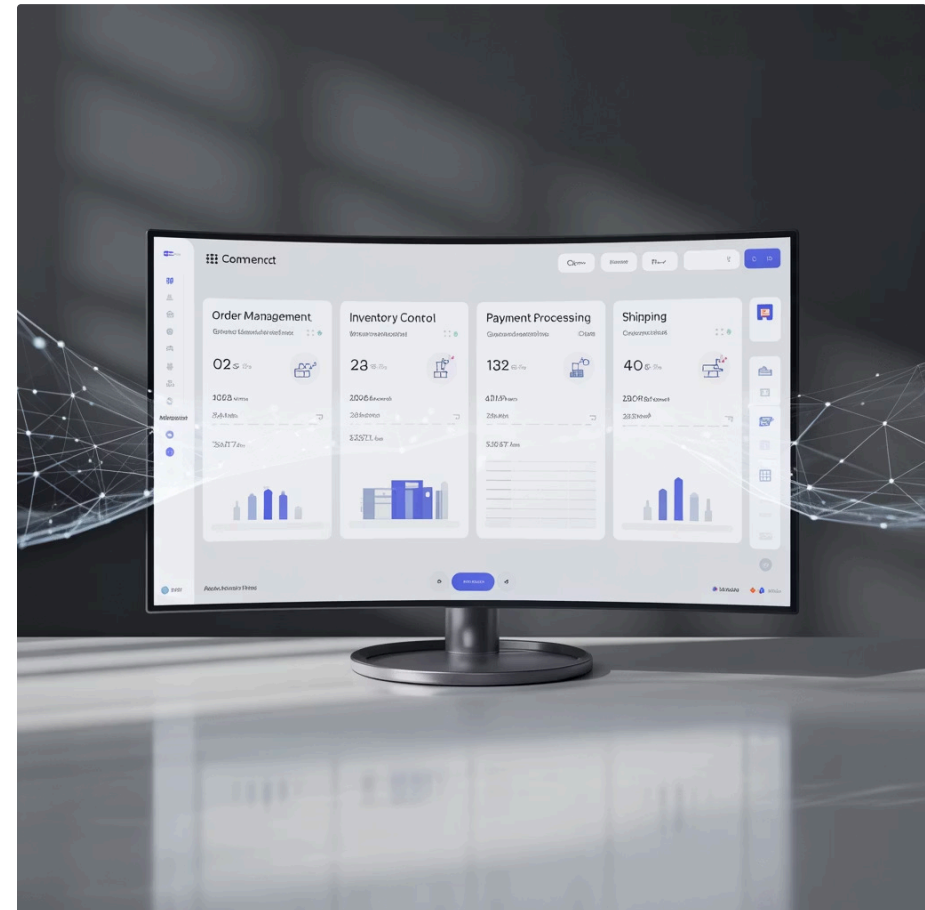
Khái niệm

Business capability là những gì doanh nghiệp làm để tạo ra giá trị (ví dụ: quản lý đơn hàng, xử lý thanh toán)

Mẫu "Decompose by business capability" khuyến nghị mỗi dịch vụ đại diện cho một khả năng kinh doanh và do một đội nhỏ chịu trách nhiệm

Khi phù hợp

Sử dụng khi mô hình tổ chức của bạn rõ ràng theo chức năng, khả năng. Các capability ổn định giúp kiến trúc ổn định và các nhóm có thể làm việc độc lập



Ví dụ: đối với cửa hàng trực tuyến, có các dịch vụ quản lý danh mục sản phẩm, quản lý tồn kho, quản lý đơn hàng và quản lý giao hàng

Phân rã theo bối cảnh giới hạn

Khái niệm

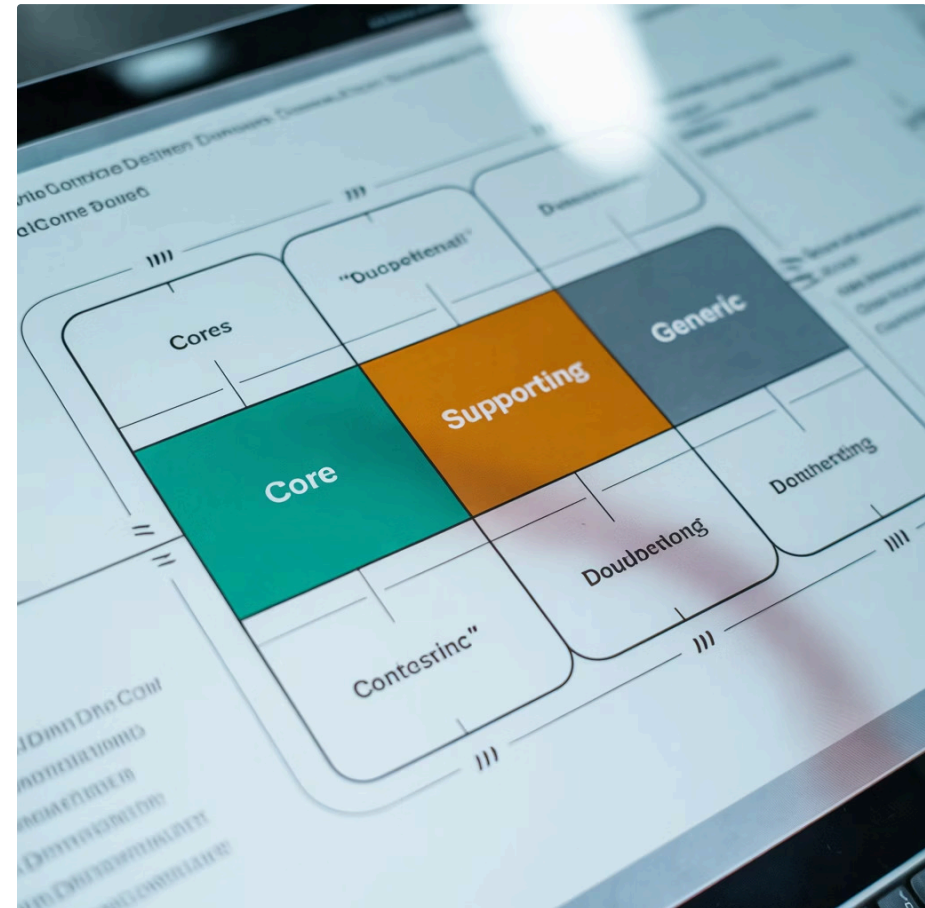
Domain-Driven Design (DDD) xác định "bounded context" – phạm vi mà một mô hình miền áp dụng

Microservices nên được thiết kế theo các subdomain của miền ứng dụng

Mẫu "Decompose by subdomain" chỉ ra rằng một domain bao gồm nhiều subdomain:

- Core (giá trị cốt lõi)
- Supporting (hỗ trợ)
- Generic (chung)

Mỗi subdomain tương ứng với một dịch vụ



Khi phù hợp

Được ưa chuộng khi sử dụng DDD, đặc biệt là những hệ thống có miền phức tạp. Phân rã theo subdomain giúp kiến trúc ổn định, dịch vụ có cohesion cao và nhóm phát triển được tổ chức quanh giá trị kinh doanh

Bối cảnh: Phân rã theo Bounded Context (DDD)

Trong Domain-Driven Design (DDD), một Domain (miền nghiệp vụ) thường rất lớn, phức tạp, chứa nhiều khái niệm, quy tắc, quy trình.

Để tránh hỗn loạn, ta chia domain thành nhiều Subdomain, mỗi subdomain bao gồm một nhóm khái niệm, quy tắc đồng nhất về ngữ nghĩa.

Mỗi Subdomain nên được triển khai thành một Bounded Context (BC) – tức “biên giới” mà trong đó mô hình nghiệp vụ được định nghĩa và áp dụng nhất quán.

👉 Nhưng không phải subdomain nào cũng quan trọng như nhau ⇒ vì thế DDD phân loại thành Core / Supporting / Generic.

Vì sao chia thành Core / Supporting / Generic?

❏

Mục tiêu:

Tập trung nguồn lực cho phần quan trọng nhất của doanh nghiệp.

Không lãng phí công sức cho những phần ít giá trị cạnh tranh (có thể outsource hoặc dùng thư viện).

Phân tách trách nhiệm: đội dev biết mình đang xây cái gì quan trọng, cái gì chỉ hỗ trợ.

Ý nghĩa từng loại Subdomain



Core Subdomain (Miền cốt lõi)

Là trái tim của business – thứ tạo ra lợi thế cạnh tranh cho doanh nghiệp.

Cần sự đầu tư kỹ lưỡng: thiết kế mô hình DDD chuẩn, code chất lượng cao, team giỏi nhất.

Thay đổi ở đây ảnh hưởng trực tiếp tới giá trị công ty.

👉 Ví dụ:

- Amazon: Engine tính giá động (Dynamic Pricing), gợi ý sản phẩm (Recommendation).
- Uber: Dispatch/Matching – thuật toán ghép tài xế-khách.
- Ngân hàng: Ledger (sổ cái) – quy tắc ghi nợ/có, đảm bảo chính xác, bất biến.



Supporting Subdomain (Miền hỗ trợ)

Cũng quan trọng, nhưng chỉ hỗ trợ cho Core.

Không phải nơi tạo khác biệt lớn, nhưng nếu bỏ đi thì Core không thể chạy.

Có thể dùng giải pháp có sẵn, hoặc viết nhưng không cần cầu kỳ bằng Core.

👉 Ví dụ:

- E-commerce: Quản lý tồn kho (Inventory), quản lý shipping.
- Uber: Quản lý hồ sơ tài xế (Driver Onboarding), xử lý bản đồ (Map/Geocoding – thường tích hợp Google Maps).
- Ngân hàng: Module KYC (xác minh khách hàng).



Generic Subdomain (Miền chung)

Là những chức năng ai cũng có, không tạo khác biệt.

Thường dùng giải pháp ngoài (off-the-shelf) hoặc framework có sẵn.

Không nên tốn công reinvent the wheel.

👉 Ví dụ:

- Auth & Identity (xác thực, phân quyền) – có thể dùng Keycloak, OAuth2 server.
- Notifications (gửi email/SMS) – tích hợp SendGrid, Twilio.
- Reporting/Logging – dùng BI tools, ELK stack.

Lợi ích của cách phân chia này

Chiến lược đầu tư:

Đầu tư nhiều vào Core, ít hơn vào Supporting, outsource Generic.

Tổ chức team:

Team A: chuyên Core (domain experts + dev giỏi nhất).

Team B: Supporting (có thể kết hợp dev và ops).

Team C: Generic (cấu hình, tích hợp giải pháp ngoài).

Tối ưu kiến trúc:

Không phức tạp hóa Generic bằng DDD, nhưng dành công sức cho Core.

Tóm gọn

Core = thứ tạo khác biệt, lợi thế cạnh tranh → tự xây, đầu tư mạnh.

Supporting = thứ cần để Core hoạt động → có thể xây hoặc mua ngoài, không cần “perfect” như Core.

Generic = thứ ai cũng cần, không khác biệt → dùng giải pháp chung, không lãng phí nguồn lực.

Cách phân rã này giúp microservices có cohesion cao, đội dev biết đâu là trọng tâm, và kiến trúc ổn định lâu dài.

Phân rã theo quyền sở hữu dữ liệu

Khái niệm

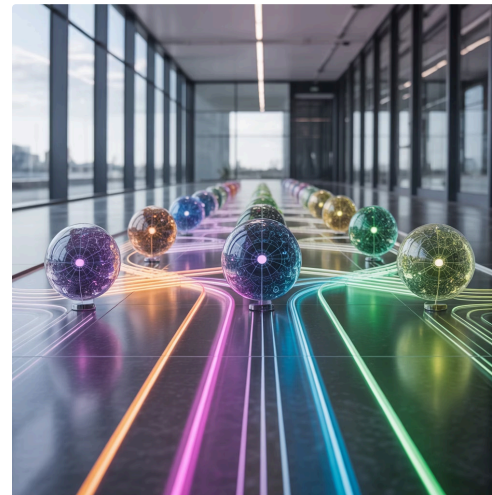
Microservices nên sở hữu dữ liệu của mình và không chia sẻ trực tiếp

Mẫu "Database per Service" giữ cho dữ liệu bền vững của mỗi dịch vụ là riêng tư

Việc truy cập dữ liệu phải qua API để tránh phụ thuộc

Hệ quả & giải pháp

Việc mỗi dịch vụ sử dụng cơ sở dữ liệu riêng giúp giảm coupling và cho phép sử dụng cơ sở dữ liệu phù hợp nhất



Thách thức khi cần giao dịch hay truy vấn dữ liệu liên quan nhiều dịch vụ:

- Không thể dùng transaction ACID truyền thống
- Phải dùng **Saga** hoặc **API composition**
- Mẫu CQRS (Command Query Responsibility Segregation)



Kích thước và số lượng dịch vụ

Các dịch vụ nên đủ nhỏ để một "đội pizza hai người" (6–10 người) có thể phát triển và quản lý

Tiêu chí xác định kích thước

- Mỗi dịch vụ nên quản lý một tính năng kinh doanh
- Được xác định thông qua bounded context
- Không tính theo số dòng code mà theo độ kết dính

Cân bằng kích thước

- Dịch vụ quá nhỏ: chi phí quản lý cao
- Dịch vụ quá lớn: mất đi lợi ích của microservices
- Cần cân nhắc chi phí vận hành và độ phức tạp

Thiết kế & ranh giới dịch vụ

Nguyên tắc trách nhiệm đơn (SRP)

Mỗi dịch vụ chỉ có một lý do thay đổi; tránh gộp nhiều chức năng khác nhau trong một dịch vụ

Định rõ API và hợp đồng

Sử dụng API rõ ràng (REST/gRPC) để giao tiếp. Thống nhất versioning và backward compatibility

Tự trị dữ liệu

Dịch vụ quản lý cơ sở dữ liệu riêng, có thể dùng RDBMS hoặc NoSQL tùy nhu cầu. Không truy cập dữ liệu dịch vụ khác trực tiếp

Phương thức giao tiếp

Sử dụng đồng bộ (HTTP, gRPC) khi cần phản hồi tức thì, và bất đồng bộ (message bus như Kafka/Service Bus) khi muốn giảm coupling và tăng khả năng chịu lỗi

Phân tách front-end

Thiết kế backends for front-ends (BFF) hoặc meta-endpoints để hỗ trợ nhiều loại client (web, mobile), tránh ghép trực tiếp client với nhiều dịch vụ nhỏ

Ưu điểm của microservices



Khả năng mở rộng linh hoạt

Mỗi dịch vụ có thể scale độc lập theo nhu cầu, tối ưu tài nguyên và loại bỏ điểm nghẽn



Triển khai và phát hành nhanh

Triển khai, rollback hoặc cập nhật một dịch vụ mà không ảnh hưởng đến phần còn lại, rút ngắn vòng đời phát hành



Đa dạng công nghệ

Mỗi dịch vụ có thể dùng ngôn ngữ và cơ sở dữ liệu khác nhau, khuyến khích đổi mới và sử dụng công cụ tối ưu



Cách ly lỗi và tăng độ tin cậy

Nếu một dịch vụ thất bại, các dịch vụ khác vẫn hoạt động; circuit breaker giúp ngăn lỗi lan rộng



Đội ngũ tự trị & code nhỏ

Mỗi dịch vụ đủ nhỏ để một nhóm nhỏ quản lý; tránh phụ thuộc chéo, tăng năng suất



Tương thích với cloud-native

Phù hợp với môi trường cloud, container và tự động hóa, sử dụng Kubernetes và orchestration

Nhược điểm của microservices

Độ phức tạp hệ thống tăng

Hệ thống phân tán có nhiều dịch vụ, mỗi dịch vụ có dữ liệu riêng, yêu cầu giải quyết giao tiếp, khám phá dịch vụ, cân bằng tải và quản lý lỗi

Chi phí vận hành

Nhiều dịch vụ dẫn tới nhiều pipeline CI/CD, nhiều container và cơ sở hạ tầng. Cần sử dụng orchestration để tự động hoá

Thử thách nhất quán dữ liệu

Mỗi dịch vụ sở hữu dữ liệu riêng nên giao dịch xuyên dịch vụ phức tạp; phải tránh 2PC và thay bằng mô hình saga

Truy vấn liên dịch vụ

Việc join dữ liệu từ nhiều dịch vụ trở nên khó khăn; giải pháp là dùng API composition hoặc CQRS

Kiểm thử và debug khó hơn

Việc kiểm thử một dịch vụ phải mô phỏng hoặc tích hợp với nhiều dịch vụ khác; yêu cầu cách tiếp cận mới

Thiếu quản trị & chuẩn hóa

Sự tự do về công nghệ có thể dẫn tới môi trường hỗn loạn. Cần áp dụng tiêu chuẩn chung về log, API và bảo mật

Biện pháp giảm thiểu rủi ro



Mô hình Saga và event-driven

Dùng mô hình Saga để xử lý giao dịch phân tán; mỗi bước là một transaction cục bộ và có transaction bồi thường khi thất bại



Giám sát và quan sát tập trung

Áp dụng log aggregation, theo dõi metrics, tracing phân tán để phát hiện và xử lý sự cố kịp thời



Orchestration và service mesh

Sử dụng Kubernetes và service mesh để tự động hóa triển khai, quản lý và bảo mật



Circuit breaker, bulkhead

Áp dụng circuit breaker để ngăn lỗi lan rộng và bulkhead để phân bổ tài nguyên riêng cho mỗi dịch vụ



Thiết lập tiêu chuẩn nội bộ

Xây dựng framework chung, định nghĩa cách đặt tên, logging, theo dõi, chính sách bảo mật và API

Trường hợp sử dụng thực tế



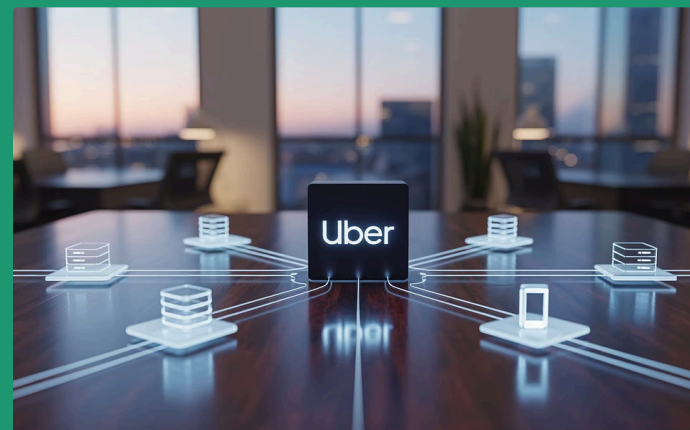
Amazon

Chuyển từ nguyên khối sang các web service đơn giản, mỗi chức năng được giao cho một đội riêng, giao tiếp qua API



Netflix

Chuyển đổi sau sự cố hỏng cơ sở dữ liệu, di chuyển lên AWS với 700+ microservices, xử lý hàng tỷ yêu cầu mỗi ngày



Uber

Tách từng chức năng thành microservices, kết nối qua API gateway, giúp triển khai nhanh hơn và dễ quản lý

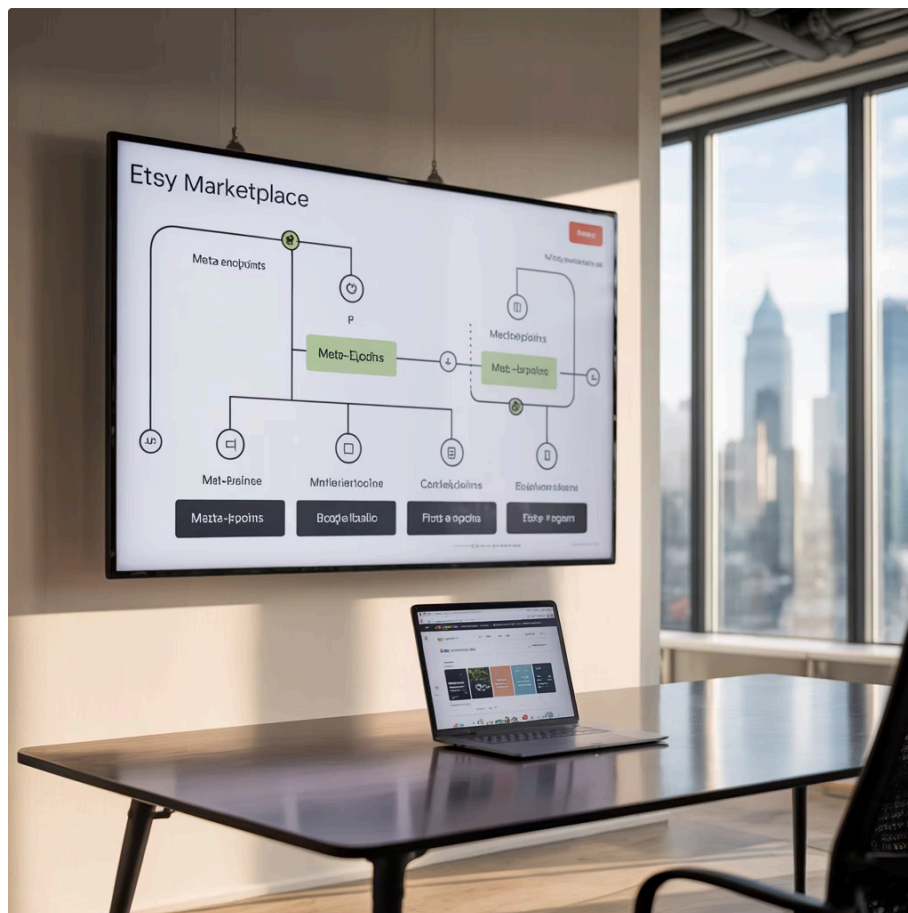
Etsy và CIBC: Trường hợp đặc biệt

Etsy – Chợ trực tuyến

Gặp vấn đề hiệu năng khi lượng truy cập tăng cao, đặt mục tiêu mỗi giao dịch dưới 1 giây

Áp dụng hệ thống API hai tầng, trong đó các meta-endpoint tổng hợp nhiều endpoint nhỏ và client chỉ gọi meta-endpoint

Sử dụng cURL và phát triển công cụ hỗ trợ gọi HTTP đồng thời, cải thiện tốc độ và dễ dàng cập nhật sản phẩm



CIBC – Ngân hàng Canada

Đối mặt với áp lực từ kênh di động, fintech và yêu cầu mở dữ liệu (PSD2)

Xác định microservices là phương pháp tốt nhất để thích ứng với tương lai biến động

Xây dựng framework microservices riêng dựa trên mã nguồn mở để kiểm soát sự thay đổi và giảm phụ thuộc vào vendor

Chứng minh microservices phù hợp với lĩnh vực ngân hàng, nơi yêu cầu độ tin cậy cao và khả năng thích ứng với quy định



Khi nào nên sử dụng microservices?

Phù hợp khi:

- Hệ thống lớn, phức tạp, cần mở rộng linh hoạt
- Tổ chức có nhiều khả năng kinh doanh độc lập
- Nhóm phát triển đa chức năng
- Cần phát hành nhanh, liên tục
- Đang chuyển đổi lên cloud
- Cần tính resilience cao
- Muốn thử nghiệm công nghệ mới
- Đáp ứng yêu cầu pháp lý thay đổi nhanh

Không phù hợp khi:

- Ứng dụng nhỏ, đơn giản
- Nhóm chưa có kinh nghiệm vận hành hệ thống phân tán
- Thiếu hạ tầng DevOps, giám sát, tự động hóa
- Không có khả năng quản trị và thiết lập tiêu chuẩn
- Chạy theo xu hướng mà không đánh giá kỹ lưỡng

Microservices không phải là "liều thuốc thần". Cần đánh giá kỹ phạm vi dự án, khả năng của đội ngũ và lợi ích mong muốn.

